Accelerating Content-Defined Chunking for Data Deduplication Based on Speculative Jump

Xiaozhong Jin, Haikun Liu, *Member, IEEE*, Chencheng Ye, *Member, IEEE*, Xiaofei Liao, Hai Jin, and Yu Zhang

***Abstract*—In data deduplication systems, chunking has a significant impact on the deduplication ratio and throughput. Existing *Content-Defined Chunking* (CDC) approaches exploit a sliding window to calculate rolling hashes of the input data stream byte-by-byte, and then determine chunk cut-points if the rolling hash satisfies a given cut-condition. Since previous CDC approaches are extremely costly, it often significantly degrades the throughput of data deduplication systems. In this paper, we argue that calculating and checking the rolling hashes byte-by-byte is unnecessary. To reduce the CPU overhead of CDC, we propose a *jump-based chunking* (JC) approach. The key idea is to introduce a jump-condition, and the sliding window can jump over a specific length of the input data stream if the rolling hashes satisfy the jump-condition. Moreover, we also explore the impact of the cut-condition and the jump-condition on the chunk size. Our theoretic studies demonstrate the effectiveness and efficiency of JC, without compromising the deduplication ratio. Experimental results show that JC improves the throughput of chunking by about 2× on average compared with the state-of-the-art CDC approaches while still guaranteeing high deduplication ratio.**

***Index Terms*—Boundary-Shift, content-defined chunking (CDC), data deduplication, rolling hash.**

I. Introduction

TODAY'S data centers are facing ever-increasing requirements of storage due to the rapid growth of data volume. Data deduplication technologies are essential to reduce the storage requirement. It eliminates redundancy by finding the same data slices and replacing the duplicate data with a reference to an existing one. According to the data granularity, deduplication can be divided into file-level and chunk-level. They both use fingerprints (fp) of files or chunks to identify the redundant data. Unlike file-level deduplication, chunk-level deduplication divides the input data into similar-sized chunks and identifies the redundancy from existing chunks in the storage. Chunk-level deduplication is more popular because it is able to find redundancy in a small granularity, and thus achieves a higher deduplication ratio.

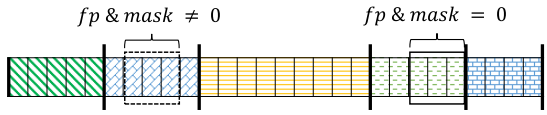


Fig. 1. Content-defined chunking with a sliding window of 3 bytes. *The data in this example is split into 5 chunks, and the cut-points are marked with bold lines ( ). On each cut-point, the fingerprint (fp) satisfies the cut-condition* $fp \,\&\, mask = 0$.

As the first step of deduplication, chunking has a significant impact on data deduplication ratio and throughput. We formalize the chunking problem as follows: given an input data S of length l, chunking algorithms split S into a number of chunks whose sizes approximate c_{avg}, which is the expected average chunk size.

Chunking algorithms can be divided into two categories: *Fixed-Size Chunking* (FSC) [1] and *Content-Defined Chunking* (CDC). FSC sequentially chunks the input into fixed sizes and has the fastest speed among various chunking algorithms. However, it cannot resist the *boundary-shift* problem [2], [3] caused by small updates, resulting in a low deduplication ratio. CDC solves this problem by chunking according to the content of the input. Most existing CDC approaches use a sliding window and chunk the input according to the fingerprint of content inside the window. As shown in Fig. 1, if the fingerprint of the sliding window satisfies a cut-condition (for example, $fp \,\&\, mask = 0$), the CDC splits the input at the position of the sliding window (called a cut-point) and generates a new chunk. Each time the sliding window moves one-byte forward, the CDC needs to recalculate the fingerprint and perform a hash judgment (i.e., check whether the fingerprint satisfies the cut-condition). To chunk 1 GB input data, CDC approaches need to calculate 10^9 fingerprints and perform 10^9 hash judgments. These procedures are time-consuming and cause heavy CPU overhead.

To reduce the calculation cost involved in CDC and speed up the chunking, we propose a *jump-based chunking* approach called JC. We propose two key technologies to achieve high throughput: 1) **Jump specific length.** Instead of sliding the window byte-by-byte, JC jumps a specific length when the fingerprint satisfies a given condition (called jump-condition). By skipping over a portion of the input, JC reduces the calculation of fingerprints and speeds up the chunking. 2) **Embedded masks.**

Manuscript received 31 January 2023; revised 17 May 2023; accepted 19 June 2023. Date of publication 29 June 2023; date of current version 20 July 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4500303, in part by the National Natural Science Foundation of China (NSFC) under Grants 62072198, 61825202, and 61929103, in part by Huawei Technologies Co., Ltd under Grant YBN2021035018A7. Recommended for acceptance by J. Carretero Perez. *(Corresponding author: Haikun Liu).*

The authors are with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, and Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: xzjin@hust.edu.cn; hkliu@hust.edu.cn; yecc@hust.edu.cn; xfliao@hust.edu.cn; hjin@hust.edu.cn; zhyu@hust.edu.cn).

Digital Object Identifier 10.1109/TPDS.2023.3290770

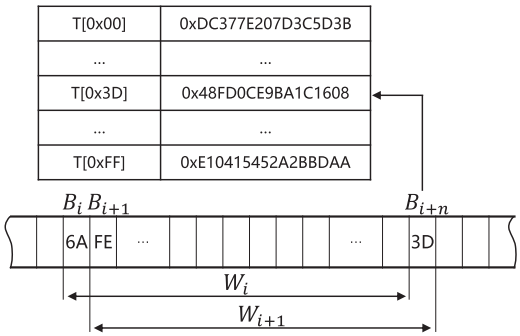


Fig. 2. Illustration of the chunking in Gear. $Gear(W_{i+1})$ *is calculated based on* $Gear(W_i)$, B_{i+n} *is equal to 0x3D.*

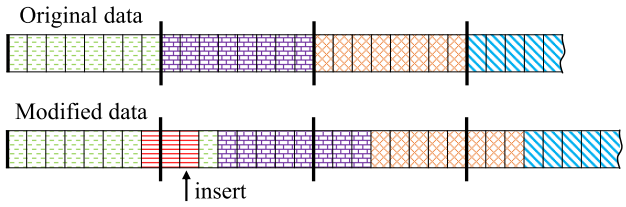


Fig. 3. Boundary-shift problem of FSC. *After inserting three bytes into the original data (marked with red), all cut-points (marked with thick lines ()) shift by three bytes.*

JC uses two masks to judge the fingerprint. By embedding one mask into another, JC reduces the cost of hash judgments and achieves higher throughput. We summarize our contributions as follows:

1) We design a new chunking approach called JC to skip over a portion of the input according to our jump-condition, and thus speed up the chunking while still achieving similar deduplication ratios compared with previous CDC approaches.
2) We explore the relationship among different parameters of JC and theoretically justify the efficiency of JC. We find that there may be a few optional settings of the jump-condition. However, different jump-conditions have little impact on the deduplication ratio and throughput.
3) We provide a theoretical proof of the throughput and deduplication ratio of JC, and also evaluate the performance of JC using 5 real-world datasets with extensive experiments. Experimental results show that JC improves the throughput of chunking by 2× on average compared with state-of-the-art CDC approaches (i.e., Rabin [4], TTTD [5], AE [6], FastCDC [3], LeapCDC [7]) while still achieving similar deduplication ratios.

The rest of paper is organized as follows. Section II introduces background and related work. Section III illustrates the detailed design of JC. Section IV presents the evaluation and we conclude in Section V.

II. Background and Related Work

In this section, we first introduce the necessary background including rolling hash (Section II-A) and boundary shift problem (Section II-B), and then introduce the related work (Section II-C).

A. Rolling Hash

Unlike traditional hash algorithms such as MD5 and SHA-1 [8], [9], [10], [11], rolling hash is a kind of hash algorithm that is used to calculate fingerprints of continuous sliding windows. For each window, it calculates a new fingerprint based on the previous one. Thus, rolling hash is able to reuse previous computations to provide rather high throughput and is widely used in CDC approaches.

Rabin Hash (Rabin) [4], [12] and Gear Hash (Gear) [13] are the most famous rolling hash algorithms. In the following, we take Gear as an example to elaborate on its implementation. Gear simplifies the Rabin by using the pre-hashed values of one-byte numbers. As shown in Fig. 2, Gear maintains a table T that holds 256 hash values of one-byte numbers. For a sliding window W_i with n bytes data from B_i to B_{i+n-1}, Gear defines the fingerprint of W_i as

$$Gear(W_i) = \left\{ \sum_{x=i}^{i+n-1} T[B_x] 2^{n-x+i-1} \right\} \ MOD\ 2^k \quad (1)$$

where k $(0 < k \leq n)$ is a predefined constant chosen according to the expected chunk size. According to the (1), we have

$$Gear(W_{i+1}) = \{Gear(W_i) \ll 1 + T[B_{i+n}]\} \ MOD\ 2^k \quad (2)$$

According to (2), Gear needs one left shift, one access to the array, one add and one *MOD* operation to get a new fingerprint. Compared with Rabin, Gear does not involve time-consuming multiplication. Thus, it reduces the amount of computations.

We note that the calculated fingerprints may frequently or rarely satisfy the cut-condition (i.e., $fp \ \& \ mask = 0$). Thus, most chunking approaches set both a lower bound c_{min} and an upper bound c_{max} to eliminate extremely small/big chunks. They do not split data when the new chunk is smaller than the lower bound, and force splitting data when the new chunk is larger than the upper bound.

B. Boundary-Shift Problem

Among all kinds of chunking approaches, FSC [1] provides the highest throughput because it splits data into chunks of the same size without incurring heavy computation. Although a smaller chunk size generally yields a higher deduplication ratio, FSC usually exhibits relatively low deduplication ratio than CDC approaches because it cannot solve the boundary-shift problem [2], [6]. As a result, FSC is typically employed in situations where optimizing throughput takes precedence over maximizing deduplication ratio.

Fig. 3 illustrates the boundary-shift problem. If an input is modified (insertion or deletion), all cut-points after the modified position shift by a fixed offset. Although there is a lot of redundancy between the original and modified data, the shifted cut-points divide the data into different chunks compared with previous chunks. Thus, deduplication systems are unable to identify redundancy in the following.

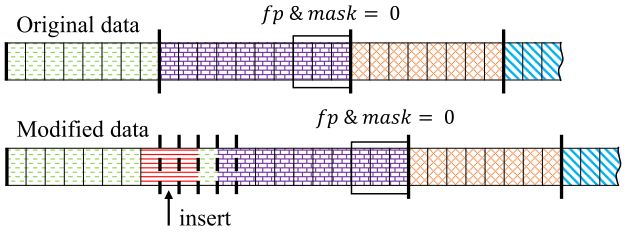


Fig. 4. Example of Gear-based CDC. *For the original input, Gear splits the data into three chunks. The cut-points are marked with thick lines. After inserting some bytes (marked with red), Gear may split chunks at any of the dotted lines (¦). However, all chunks after the purple chunk keep the original boundaries.*

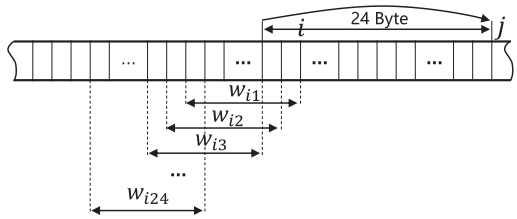


Fig. 5. Illustration of LeapCDC. *For each byte, LeapCDC defines 24 continuous windows. If any window does not meet the pre-defined cut-condition (e.g., the third window w_{i3}), it jumps 24 B forwards (from the position $i-2$ to $i+22$, i.e., j) and then repeats this process.*

CDC addresses this problem by chunking the input according to its contents. Taking the Gear-based CDC approach as an example, Fig. 4 illustrates that CDC can still correctly identify most previous cut-points. When a piece of new data is inserted into the input, Gear may split chunks around the inserted position according to the content of the data. However, the right boundary of the purple chunk is not changed, and all chunks after the orange one keep the original boundaries. Thus, a modification (inserting, deleting or overwriting) only affects a limited number of chunks, and most of the chunks keep unchanged. Compared with FSC, CDC approaches improve the deduplication ratio.

However, because CDC approaches need to calculate the rolling hashes of the input data byte-by-byte, the chunking throughput is usually very low. We argue that the byte-by-byte calculation is unnecessary, and a sophisticated CDC algorithm is able to reduce the computation overhead while still guaranteeing a high deduplication ratio. It is potential to jump a portion of the input speculatively while still addressing the boundary-shift problem (Section III-D).

C. Related Work

In the following, we first introduce the CDC approaches based on rolling hash. To make an even distribution of chunk sizes, TTTD [5] is based on Rabin Hash, and uses a normal cut-condition and a backup cut-condition to determine the boundary of a chunk. The backup cut-condition is easier to meet than the normal one. When the window slides, each fingerprint fp is checked with the two cut-conditions. When the offset of the sliding window exceeds the maximum chunk size and the normal condition is still not met, the last position that satisfies the backup condition is set as a cut-point. FastCDC [3] uses a stricter cut-condition when the chunk size is small, and changes to a looser condition when the chunk size is big. In this way, it normalizes the chunk sizes in a small but dense distribution. RapidCDC [14] predicts the chunk size by exploring the locality of duplication. It argues that if two files share the same chunk, the following content of the two files are most-likely identical. Based on this assumption, RapidCDC follows previous cut-points to chunk the new input, and thus reduces the calculation. Since modern deduplication systems [15], [16] performs the chunking, indexing, and storing in a pipeline for higher throughput, while RapidCDC has to known whether the previous chunk is redundant before calculating the next chunk, it may stall the pipeline and limits the system's throughput. Because rolling-hash algorithms need to generate a fingerprint for each sliding window, they are often costly for a large volume of input. To reduce these calculations, JC speculatively jumps over a specific length of the input to speed up the chunking.

Unlike fingerprint-based chunking, MAXP [17] splits chunks according to the extreme value of the input. It sets up a symmetric sliding window and sets the midpoint as a cut-point if the midpoint is the extreme of the window. Thus, MAXP needs more than six condition judgments for each sliding window. AE [6] improves MAXP by searching the extreme value in an asymmetrical range and reducing operations needed by each sliding window to 2 condition judgment. These CDC approaches show similar performance to the rolling hash based chunking and are orthogonal to JC.

To reduce the cost of fingerprinting, Yu et al. [7] propose leap-based CDC (LeapCDC). As shown in Fig. 5, at each position i, leap-based CDC defines n continuous windows w_{ij} ($1 \le j \le n$), and a typical value of n is 24. From the window w_{i1} backward to w_{i24}, it performs a pseudo-random transformation on each window. The pseudo-random transformation is implemented with a predefined table E. For each window w_{ij}, LeapCDC chooses 5 bytes $B_1, \ldots, B_5$ from w_{ij}. w_{ij} satisfies the cut-condition if $E[B_1] \oplus E[B_2] \oplus E[B_3] \oplus E[B_4] \oplus E[B_5] \neq 0$. LeapCDC sets the current point i as a cut-point only when all these consecutive n windows satisfy the cut-condition. If any window w_{ij} does not meet the condition, the following $n-1$ bytes of input is not a cut-point definitely. LeapCDC moves the current position (i.e., $i-j+1$) forward n bytes and then repeats this procedure. Thus, LeapCDC calculates the pseudo-random transformation from right to left while jumps from left to right, and spirals forward. In this way, LeapCDC accelerates chunking by jumping over positions that are not cut-points.

However, LeapCDC has two major disadvantages. First, the pseudo-random transformation in LeapCDC is slower than AE and Gear. Since LeapCDC involves frequent jumps (jumps every 20 bytes on average, see Section III-C) and destroys the continuity of the data, it is unable to use more efficient rolling hash algorithms like Rabin and Gear. Moreover, because each pseudo-random transformation needs 5 random accesses to the table E and 4 XOR operations, LeapCDC is often costly due to poor data locality. Second, it is hard to tune the performance of LeapCDC. Because the chunk size is correlated with many other parameters such as the table E and the number of windows n, it is difficult to orchestrate them together to achieve the best

performance. LeapCDC only skips positions that are definitely not cut-points. In contrast, JC speculatively jumps over a long piece of the input infrequently if the sliding window satisfies a jump-condition. Because the infrequent jumping does not break the continuity of sliding windows, JC can still exploit the rolling hash to further improve the throughput of chunking relative to LeapCDC. Our theoretical and experimental studies demonstrate that our speculative jumps have little impact on the deduplication ratio.

We summarize the characteristics of different chunking approaches as follows. FSC achieves the highest throughput [1] but provides limited deduplication ratio. Rabin, Gear, TTTD, FastCDC, LeapCDC, and AE all achieve high deduplication ratios, but suffer from low throughput. Moreover, it is difficult to tune the performance of LeapCDC. In contrast, JC provides both high throughput and high deduplication ratio.

Recently, there have been a number of systematical studies on the acceleration of data deduplication. For example, some proposals exploit multiple-threading and pipelining technologies [15], [16], [18] to parallelize different deduplication stages, and thus improve the throughput of deduplication systems. A few works exploit heterogeneous computing resources [19], [20] such as GPUs/FPGAs to accelerate data deduplication. CROCUS [19] orchestrates CPUs and GPUs to enhance the throughput of deduplication systems. REDUP [21] provides a deduplication-aware hierarchical caching mechanism to accelerate I/O operations for machine learning frameworks. These technologies are orthogonal to the proposed speculative jump. Since content-defined chunking is a performance bottleneck of data deduplication, JC can exploit these advanced technologies to improve the throughput of chunking.

III. Design

In this section, we present key designs of JC and verify its efficiency theoretically and experimentally.

A. Overview

As mentioned in Section II, it is unnecessary to slide the window and calculate the fingerprints byte-by-byte. Thus, we propose JC, a jump-based chunking approach that aims to provide high throughput while guaranteeing a high deduplication ratio. There are mainly two key designs in JC, i.e., speculative jumping and embedded masks. First, JC allows the sliding window to jump forward a specific length by introducing a jump-condition. Second, JC embeds one mask into another to reduce the number of hash judgments, and thus mitigate the CPU cost for condition judgements.

Fig. 6 illustrates the whole flow of JC-based data deduplication. First, files are partitioned into chunks of approximately the same size via JC. Second, the fingerprint of each chunk is generated using strong hashing algorithms such as SHA-1, MD5, or xxHash [22]. Third, each fingerprint is searched in a global hash table to identify whether the content of the chunk is the same with an existing one. These chunks are indexed based on their fingerprints. Finally, only unique chunks are kept in storage, and other duplicate chunks are replaced by references.

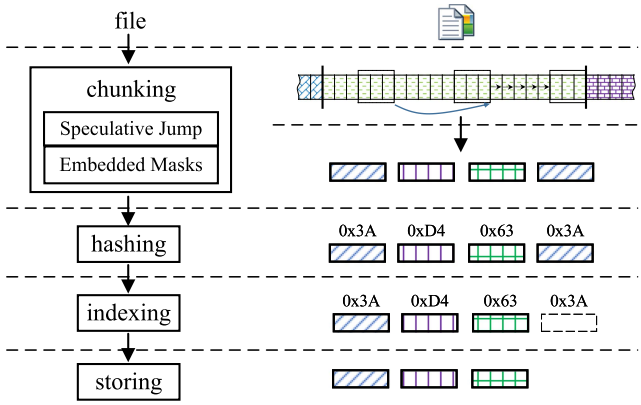


Fig. 6. Procedure of data deduplication.

TABLE I
Summary of Notations

Notation	Description
S	the data to be deduplicated
l	the length of S
fp	fingerprints
c_{min}	the minimum chunk size
c_{avg}	the average chunk size
c_{max}	the maximum chunk size
j_s	the length skipped by a single jump
j_{avg}	the average length skipped in a chunk
$maskC$	the mask of cut-condition
$maskJ$	the mask of jump-condition
$cOnes$	the number of "1" contained in the binary representation of maskC
$jOnes$	the number of "1" contained in the binary representation of maskJ
p_c	the probability that a sliding window satisfies the cut-condition
p_j	the probability that a sliding window satisfies the jump-condition

The key designs of JC answer four important questions. How to reduce unnecessary calculation of rolling hashes in CDC algorithms? How to reduce the cost of condition judgements raised in our new algorithm? Can we theoretically validate the efficiency of our new algorithm? Whether the proposed speculative jump affects the deduplication ratio?

B. Key Designs of JC

In the following, we present two key designs of JC, i.e., jumping on specific condition and embedded masks.

1) Jumping on Specific Condition: To reduce the cost of rolling hash, we propose a new condition (called jump-condition) to jump a specific length when the sliding window is moving forward. JC uses two masks, namely $maskC$ and $maskJ$ in Table I, to determine the cut-point of the input data. When $fp \,\&\, maskC = 0$, JC sets the current position of the sliding window as a cut-point and generates a new chunk. When $fp \,\&\, maskJ = 0$, JC jumps forward a pre-defined length. The cut-condition or jump-condition requires that all bits in the fingerprint should be 0 if the corresponding bits in the $maskC$ or $maskJ$ are "1". For example, when the mask is "0b1111010000", the fingerprint satisfies the cut-condition only

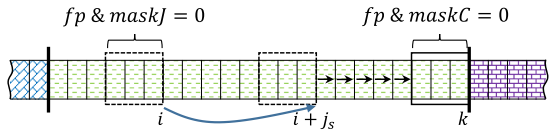


Fig. 7. Illustration of JC. *JC jumps over j_s bytes when the sliding window satisfies the jump-condition (i.e., fp & $maskJ = 0$), and then continues to calculate rolling hashes after the jump.*

if its 5th, 7th, 8th, 9th and 10th bits (from right to left) are equal to 0. Assuming the input is random [6], [17], since the hash values are uniformly distributed, each bit in the fingerprint has a probability of $\frac{1}{2}$ to be 0 or 1. Let $cOnes$ be the number of "1"s in the binary $maskC$ and p_c be the probability that a given fingerprint satisfies the cut-condition, we have

$$p_c = \left(\frac{1}{2}\right)^{cOnes} \tag{3}$$

Since it is costly to slide the windows per byte and calculate their fingerprints, JC jumps over a pre-defined length j_s when the fingerprint satisfies the jump-condition. Let $jOnes$ be the number of "1"s contained in $maskJ$ and p_j be the probability that a given fingerprint satisfies the jump-condition, we have

$$p_j = \left(\frac{1}{2}\right)^{jOnes} \tag{4}$$

The chunking procedure of JC is shown in Fig. 7. When the fingerprint in the position i meets the jump-condition, the window jumps forward by j_s bytes to the position $i + j_s$ and continues to slide forward. For example, assume that $maskJ$ is set to "0b1111000000" and the sliding window in the position i contains three bytes, 0x7D, 0x7B, 0x8C. The fingerprint (i.e., SHA-1 value) of these three bytes is 0xf2b97337 and satisfies the jump-condition (i.e., 0xf2b97337 & $maskJ = 0$). Then, JC jumps over j_s bytes and directly calculate the rolling hash at the position $i + j_s$. When the fingerprint of the sliding window at the position k meets the cut-condition (i.e., fp & $maskC = 0$), JC splits the input and generates a new chunk. In order to make an even distribution of the chunk sizes, $jOnes$ is usually set to be smaller than $cOnes$. The smaller $jOnes$ makes the jump-condition easier to be met than the cut-condition. Thus, most chunks have met at least one jump-condition before meeting the cut-condition and splitting the input, i.e., most chunks contain a portion of data that can be jumped over.

As shown in Table I, let j_s be the length of the input skipped by a single jump, j_{avg} be the average length jumped in a single chunk, c_{avg} be the expected chunk size on average, and l be the length of the input. Thus, the number of chunks is $\frac{l}{c_{avg}}$, and the total length jumped of the whole input S is $\frac{l}{c_{avg}} j_{avg}$. The expected number of chunks equals the number of cut-points, which in turn equals the product of the length of data that has not been jumped and the probability of a sliding window that satisfies the cut-condition. Thus, we have

$$\frac{l}{c_{avg}} = \left(l - \frac{l}{c_{avg}} j_{avg}\right) \times p_c \tag{5}$$

After we simplify (5), we get

$$c_{avg} = j_{avg} + \frac{1}{p_c} \tag{6}$$

In a single chunk, the average jumped length j_{avg} equals the product of the jump-length j_s and the expected number of jump-points, which in turn equals $(c_{avg} - j_{avg})p_j$. Thus, we get

$$j_{avg} = j_s(c_{avg} - j_{avg})p_j \tag{7}$$

According to (6) and (7), we get

$$j_{avg} = j_s \frac{p_j}{p_c} \tag{8}$$

Thus, the average length that has not been jumped over inside a chunk becomes

$$c_{avg} - j_{avg} = c_{avg} - j_s \frac{p_j}{p_c} \tag{9}$$

Insight: JC jumps over a specific length in a input data to speed up the chunking. To get a chunk with an expected size c_{avg}, JC only needs to calculate $c_{avg} - j_s \frac{p_j}{p_c}$ fingerprints on average rather than c_{avg} fingerprints.

2) Embedded Masks: Although jumps can reduce the calculation of fingerprints, it introduces an extra hash judgment for the jump-condition. JC needs to execute two hash judgments for each sliding window, i.e., 1) whether it satisfies the cut-condition and 2) whether it satisfies the jump-condition. As mentioned in FastCDC [3], massive hash judgments are a performance bottleneck of state-of-the-art chunking algorithms, and newly-introduced judgments of jump-conditions have a non-trivial impact on the performance. To solve this problem, we design embedded masks to mitigate the computing overhead.

Previous studies [3], [4], [13] only consider the number of "1"s in the binary representation of the mask, while we take the positions of "1"s into account in our embedded masks. As shown in Algorithm 1, we construct a bigger $maskC$ to overlap a smaller $maskJ$. In other words, by changing some "1"s in the $maskC$ into "0"s, $maskC$ could become $maskJ$. Thus, fp & $maskC$ may equal 0 only when $fp\&maskJ = 0$. In this way, the judgment of the cut-condition could be embedded into the judgment of the jump-condition (lines 11-16). Because most fingerprints do not satisfy these two conditions, only the outer judgment is executed for these fingerprints (line 11). Thus, with these embedded masks, JC can reduce two hash judgments to one for most fingerprints and speed up the chunking. The fingerprints that satisfy the jump-condition still should be checked with the inner judgment (line 12). However, since they only account for a small percentage (1/2048 under the chunk size of 8 K), they have a trivial impact on the performance.

Since the $maskC$ overlaps $maskJ$, if a fingerprint satisfies the cut-condition, it also satisfies the jump-condition. Thus, p_j, i.e., the probability that a fingerprint only satisfies the jump-condition becomes

$$p_j = \left(\frac{1}{2}\right)^{jOnes} - \left(\frac{1}{2}\right)^{cOnes} \tag{10}$$

According to (3), (6) and (10), we set $cOnes$, $jOnes$, and j_s to $log_2 c_{avg} - 1$, $log_2 c_{avg} - 2$, and $c_{avg}/2$, respectively, where

Algorithm 1: JC chunking algorithm.

```
Function jumpChunking(src, size)
    Data: src ← buffer, size ← data size,
          c_min ← minimumChunkSize,
          c_max ← maximumChunkSize;
    Result: cut-point;
    fp ← 0, j_s ← jumpLength, i ← m ;
    size ← min(c_max, size);
    /* maskC contains 11 non-zero bits          */
    maskC ← 0x590003570000;
    /* reset the least non-zero bit in maskC    */
    maskJ ← 0x590003560000;
    if size < c_min then
        return size;
    end
    while i < size do
        fp ← hash(slideWindowAtPosition(i));
        if fp & maskJ = 0 then
            if fp & maskC = 0 then
                return i;
            end
            fp = 0;
            i = i + j_s;
        end
        i++;
    end
    return min(i, size);
end
```

c_{avg} is usually set to 4 KB or 8 KB. We use this set of parameters as the default configuration of JC because they can maximize the length of the jump.

We note that JC is independent of hash algorithms, and can be integrated with different algorithms such as Rabin Hash and Gear Hash. JC uses Gear as the default hash algorithm for efficiency.

Insight: By embedding one mask into another, the two condition judgments can be nested and the number of hash judgments needed by each fingerprint is reduced from two to one.

C. Efficiency of JC

In this section, we theoretically analyze the chunking efficiency of JC and compare it with two state-of-the-art algorithms, namely Gear Hash and LeapCDC. Given an input of length l, the efficiency of chunking approaches are attributed to two aspects: 1) the number of fingerprints the chunking approach needs to calculate, 2) the latency of calculating a single fingerprint.

Most CDC approaches such as Rabin and Gear slide the window byte-by-byte and thus need to calculate l fingerprints. For JC, it jumps j_{avg} bytes of the input on average for each chunk of c_{avg} bytes. According to (3), (9), and (10), given an input of length l, JC needs to calculate $\frac{c_{avg}-j_{avg}}{c_{avg}} l = 0.5\ l$ fingerprints under our default configurations. Since JC still uses Gear to calculate the rolling hashes for sliding windows that are not skipped, it can improve the chunking throughput by about $2\times$.

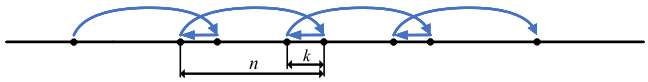


Fig. 8. Leaping forward and sliding backward in LeapCDC.

TABLE II
OPERATIONS REQUIRED TO CALCULATE A NEW FINGERPRINT IN DIFFERENT CHUNKING APPROACHES

Approaches	Operations
Rabin	2 additions, 2 multiplications, and 1 MOD
Gear	1 shifting, 1 addition, 1 MOD, and 1 array access
LeapCDC	5 array accesses and 4 XORs

For LeapCDC, the number of pseudo-random transformations is determined by p_c and n, i.e., the probability that a random window w_{ij} met the cut-condition, and the number of windows w_{ij} that needs to satisfy when the given position i is set as a cut-point. As shown in Fig. 5, at each position i, LeapCDC contains $n = 24$ windows, if window w_{i2} (position $i-1$) does not meet the cut-condition, LeapCDC moves the current position 24 bytes forward from $i-1$ to j. For each position i, let p_{ij} be the probability that LeapCDC jumps forward at window w_{ij} ($1 \leqslant j \leqslant n$), it requires $j-1$ windows before w_{ij} meets the cut-condition, and w_{ij} does not meet the cut-condition. Thus, we have

$$p_{ij} = (1-p_c)p_c^{j-1} \tag{11}$$

Let k be the average number of pseudo-random transformations calculated at position i, and then we have

$$k = \sum_{j=1}^{n} j \times p_{ij} \tag{12}$$

According to LeapCDC [7], n is set to 24 and p_c is $3/4$ by default. Based on (11) and (12), $k = 3.97$. Fig. 8 illustrates that LeapCDC leaps forward n bytes and then slides backward k bytes. For each $n-k$ bytes of the input, LeapCDC has to calculate k pseudo-random transformations. Given an input of length l, LeapCDC needs to calculate $\frac{k}{n-k} l = 0.2\ l$ fingerprints.

Although LeapCDC calculates fewer fingerprints than JC, it involves more memory accesses during the fingerprint calculation. We calculate the time consumption of JC and LeapCDC based on Intel's user manual [23]. The normalized latencies of addition, bit-shifting, and MOD are 1, 1, 4, respectively, and the normalized latency of XOR with memory accesses is 7. Table II lists the operations needed to produce a new fingerprint for different chunking approaches. Given an input of length l, the time consumption of JC is $(1 \times 1 + 1 \times 1 + 4 \times 1) \times 0.5\ l = 3\ l$, and the time consumption of LeapCDC is $(7 \times 5) \times 0.2\ l = 7\ l$ ($2.3\times$ higher than JC). We also measure the throughput of different chunking approaches in Section IV-B, and our experiments demonstrate similar results with the theoretical analysis.

Insight: For a given dataset, JC can reduce the calculation of fingerprinting by 50% compared with Gear-based CDC. Since

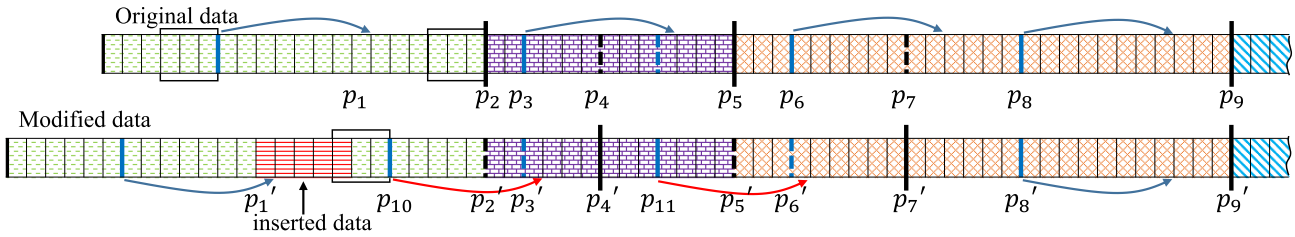


Fig. 9. Illustration of data insertion in JC. *All jump-points are marked as bold blue lines (), and all cut-points are marked as bold black lines (). The points that satisfy the jump-condition or the cut-condition but reside in the jumped (purple) area are marked with blue and black dotted lines (and), respectively. After inserting a piece of new data (red), the jumped area may continuously skip over the original jump/cut-points. For example, the new jump from p10 skips over the old cut-point p2′ and jump-point p3′. Once the new chunking process finds a cut/jump-point in the original data (p8′), the chunking process after the point will keep the same as the original chunking process.*

JC is able to leverage the rolling hash, it can also reduce the cost of fingerprinting by 57% compared with LeapCDC. JC is about 2× and 2.3× faster than Gear-based CDC and LeapCDC.

D. Effectiveness of Solving Boundary-Shift Problems

In this section, we explore the impact of JC on the boundary-shift problem and the deduplication ratio. Fig. 9 illustrates how the boundary-shift problem is potentially caused by JC. JC splits the original data into four chunks marked with different fill patterns. The sliding window is represented by a black rectangle, and its last byte is considered as a cut-point or a jump-point if the fingerprint satisfies $fp \,\&\, maskC = 0$ or $fp \,\&\, maskJ = 0$, respectively. When we insert 5 bytes of data into the first chunk near the position p_1, the inserted data may introduce a new jump-point p_{10}. After the jump, the sliding window skips over the original cut-point p_2 and the original jump-point p_3. When the window continues to slide, JC finds a cut-point p'_4 in the original jumped area and splits the original purple chunk. Later, a new jump-point p_{11} is found, and the original cut-point p_5 and jump-point p_6 are skipped. Since the data in the jumped area is not checked entirely, the following (orange) chunk may be split differently from the original data.

However, once a jump-point or cut-point in the modified data is found at the same position of the original data, the following chunking will be performed in the same way as the original chunking. In Fig. 9, assume p_8 is a jump-point that can be found by JC in both the original data and the modified data, all chunks after p_8 (from the blue chunk to the end) would keep the same as the original chunks. We define Affected Length (**AL**) as the length from the end of the inserted data to the first identical chunk so that the remaining data can be chunked identically with the original data. A shorter AL implies a higher deduplication ratio. In Fig. 9, the AL begins at p'_1 and ends at p'_9, and thus equals 3 chunks.

When the data is chunked to size c_{avg}, given a wrong split data slice (for example, p'_2 to p'_5 in Fig. 9), to make AL increase 1 chunk, JC needs to jump over the original cut-point (i.e., p'_5). Thus, it requires two necessary conditions, 1) there is at least one jump-point in the original chunking process (p_3), 2) the jumped area of the original chunking process contains at least one jump-point (p_{11}). The probability of condition 1 is $P_{C1} = 1-(1-p_j)^{c_{avg}-c_{min}-1}$, and the probability of condition 2 is $P_{C2} = 1-(1-p_j)^{j_s}$. If we set c_{avg} to 4 K, c_{min} to 512 bytes, under the default configuration, $P_{C1} = 0.826$ and $P_{C2} = 0.632$. Note that these two conditions are the necessary and insufficient conditions for the growth of AL. The probability that AL is bigger than 1 is less than $0.826 * 0.632 = 52.2\%$, and the probability that AL is bigger than 4 is less than $0.522^4 = 7.4\%$. Therefore, the average length of AL is rather short.

TABLE III
AFFECTED LENGTH (AL) OF DIFFERENT CHUNKING APPROACHES FOR TWO REAL-WORLD DATASETS

Data-sets	Chunking appraoches	AL (chunks)				
		1	2	3	4+	average
GCC	Rabin	40.22%	20.54%	10.96%	28.27%	2.27
	TTTD	39.79%	21.59%	11.41%	27.21%	2.26
	Gear	43.93%	20.78%	10.93%	24.26%	2.16
	LeapCDC	36.55%	20.31%	11.18%	31.95%	2.39
	JC	43.05%	19.68%	10.14%	27.12%	2.21
VIM	Rabin	48.00%	21.51%	12.80%	17.60%	2.64
	TTTD	46.10%	25.38%	12.14%	16.30%	2.57
	Gear	48.29%	23.33%	12.14%	16.15%	2.53
	LeapCDC	37.84%	23.97%	12.16%	26.03%	3.29
	JC	49.38%	21.12%	11.83%	17.58%	2.61

To verify our theoretical inference, we measure the length of AL using two real-world datasets. We deduplicate two consecutive versions of GCC [24] and Vim editor [25], respectively. There are often lots of difference between two adjacent versions due to modifications such as insertions, deletions and overwrites, and we measure the AL caused by these modifications. As shown in Table III, about 70% of modifications only affects the first three chunks. All chunking approaches have similar AL on average. JC affects 2.61 chunks at most, and is only 0.08 higher than Gear which has the smallest AL. For a typical 2 MB file, the extra 0.08 chunks caused by JC would only decrease the deduplication ratio by 0.02% (4KB*0.08/2 MB). Although existing proposals [6], [17] assume the input of chunking is random, most datasets have their special data distribution patterns which can be deemed as a noise of the ideal random distribution. The noise may have a similar impact on the deduplication ratio. Thus, the 0.02% degradation of deduplication ratio caused by JC is trivial (Section IV-D).

Like other chunking approaches, JC has little impact on the AL because both p_j and p_c are small, and the inserted data can hardly change the old cut-point and jump-point. Thus, the situation described in Fig. 9 is rare. *Overall, our theoretical*

analysis and experimental studies all verify that the average AL in JC is rather short, and has little impact on the deduplication ratio.

E. Distribution of Chunk Sizes

In this section, we analyse the impact of the distribution of chunk sizes on the deduplication ratio. We also discuss the expectation of the average chunk size because both c_{min} and c_{max} have a non-trivial impact on it, and allow it deviate from the expected average value c_{avg}. Like most chunking approaches, JC restricts the size of a chunk within the range of (c_{min}, c_{max}) [3], [5], [26], [27]. It does not split data into chunks when the data size is less than c_{min}, but forces to split data into chunks when the data size exceeds c_{max}. By default, we set c_{min} to 512 bytes, c_{max} to $2c_{avg}$, and j_s to $c_{avg}/2$. Thus, given a chunk smaller than c_{max}, there are at most 3 jumps inside the chunk.

Given an input data with a size of l, let x be the index of the input. If the chunking approach sets x as a cut-point and there are k $(0 \leq k \leq 3)$ jumps before position x, x should satisfy the following conditions: k bytes at the jump-position satisfies the jump-condition, other data from c_{min} to $x-1$ satisfy neither jump-condition nor cut-condition, and the byte x satisfies the cut-condition. Let $p_k(x)$ be the probability of this case, we have

$$p_k(x) = C^k_{(x-c_{min}-k\cdot j_s-1)} p_j^k \times (1-p_c-p_j)^{(x-c_{min}-k\cdot j_s-k-1)} p_c \quad (13)$$

Let $p(x)$ be the total probability of chunking the input at the position x (i.e., cut-point). Since c_{min} is the minimum chunk size, when $x < c_{min}$, $p(x) = 0$. When $c_{min} \leq x < j + c_{min}$, there would be no jumps between the position 1 and x, and thus $p(x) = p_0(x)$. When $j + m \leq x < j + 2\,m$, there may be zero or one jump between the position 1 and x, and thus $p(x) = p_0(x) + p_1(x)$. When $j + 2\,m \leq x < j + 3\,m$, there may be zero, one or two jumps between the position 1 and x, and thus $p(x) = p_0(x) + p_1(x) + p_2(x)$. Overall, the value of $p(x)$ can be summarized as follow:

$$p(x) = \begin{cases} 0 & (1 \leq \mathrm{x} < \mathrm{c_{min}}) \\ p_0(x) & (\mathrm{c_{min}} \leq \mathrm{x} < \mathrm{c_{min}} + \mathrm{j}) \\ \sum_{j=0}^{1} p_j(x) & (\mathrm{c_{min}} + \mathrm{j} \leq \mathrm{x} < \mathrm{c_{min}} + 2\mathrm{j}) \\ \sum_{j=0}^{2} p_j(x) & (\mathrm{c_{min}} + 2\mathrm{j} \leq \mathrm{x} < \mathrm{c_{min}} + 3\mathrm{j}) \\ \sum_{j=0}^{3} p_j(x) & (\mathrm{c_{min}} + 3\mathrm{j} \leq \mathrm{x} < \mathrm{M}) \\ 1 - \sum_{j=1}^{x-1} p(j) & (\mathrm{x} = \mathrm{c_{max}}) \end{cases} \quad (14)$$

Similarly, we can analyze other CDC approaches. Taking Gear as an example, if we set the position x as a cut-point, the input from the c_{min}th byte to the $x-1$th byte should not satisfy the cut-condition, and the xth byte satisfies the cut-condition. The $p(x)$ in Gear can be calculated as follows.

$$p(x) = \begin{cases} 0 & (1 \leq \mathrm{x} < \mathrm{c_{min}}) \\ (1-p_c)^{x-c_{min}-1} p_c & (\mathrm{c_{min}} \leq \mathrm{x} < \mathrm{c_{max}}) \\ 1 - \sum_{j=1}^{x-1} p(j) & (\mathrm{x} = \mathrm{c_{max}}) \end{cases} \quad (15)$$

Based on (15) and (16), the expected average chunk size c_{avg} can be calculated by $\sum_{x=m}^{c_{max}} xp(x)$. When we set the chunk size to 8 KB, the expected average chunk size of Gear and JC is 7524 bytes and 7236 bytes, respectively, smaller than 8 KB. The reason is that about 15% chunks that are larger than the defined c_{max} are truncated compulsorily, lowering the average chunk size.

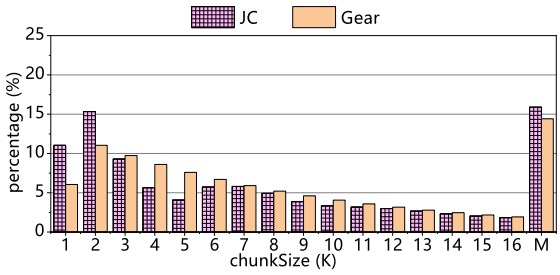


Fig. 10. Distribution of chunk size of JC and Gear when set c_{avg} to 8 KB.

Fig. 10 shows the distribution of chunk sizes when the estimated average chunk size is set as 8 KB. The last bar (M) shows the percentage of chunks that are larger than the defined c_{max}. JC introduces a few more forced truncations of large chunks than Gear. However, forced-truncation does not necessarily affect the deduplication ratio. Under the same input, if the chunking approach forces the truncation at the same position, the truncated chunk will still be deduplicated. As evaluated in Section IV-D, JC and Gear show a similar deduplication ratio.

Insight: Although JC may slightly affect the distribution of chunk sizes and the average chunk value, such variations have little impact on the deduplication ratio.

IV. Evaluation

In this section, we evaluate the performance of JC in terms of chunking throughput, CPU consumption, and deduplication ratio compared with state-of-the-art approaches.

A. Experiment Setup

System Setup: We evaluate JC on a system equipped with two 2.2 GHz Intel Xeon Gold 5220 CPUs, 64 GB DDR4 main memory, and 500 GB HP EX900 SSD. The operating system is Ubuntu 20.04.2 LTS with kernel version 6.13.0. The file system is EXT4 mounted with *rw, relatime* options. Unless otherwise specified, JC sets $cOne$ to $log_2(c_{avg}) - 1$, $jOne$ to $log_2(c_{avg}) - 2$, and j_s to $c_{avg}/2$ as a default configuration.

State-of-the-arts for Comparison: We use the destor [15], [16] framework to measure the performance of JC. Destor supports several typical chunking approaches and has been used by many previous studies [28], [29], [30], [31]. We use Gear [13] as the hash algorithm of JC and use xxHash [22] to index chunks for high throughput. We compare JC with state-of-the-art chunking approaches, including Rabin [4], TTTD [5], AE [6], Gear [13], fastCDC [3], and leap-CDC [7]. Each experimental result is the average value of three trials.

Datasets: In order to evaluate the performance of JC in different scenarios, we use five typical datasets:

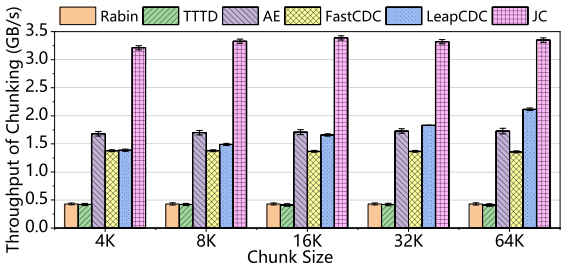


Fig. 11. Throughput of various chunking approaches. *The standard deviations are less than 6 MB/s.*

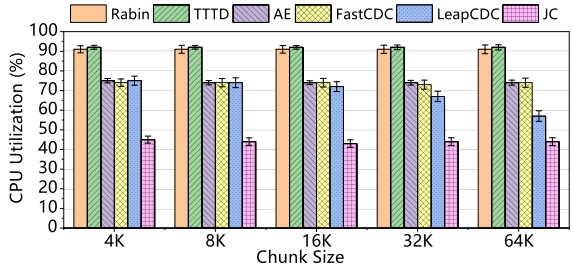


Fig. 12. CPU utilization of various chunking approaches. *The standard deviations are below 3%.*

1) *DOC:* DOC dataset contains about 2 GB of PDF papers and PPT slides. All documents are downloaded from the network. There may be substantial redundancy among these papers and slides.
2) *NEWS:* NEWS contains 4.7 GB snapshots of BBC news in 5 consecutive days. We download these web pages using *wget* with a recursion depth of 2.
3) *TAR:* This dataset contains 42 GB GCC sources with 108 different versions in the tar format. The data is downloaded from the official website of gcc [24].
4) *VMI:* This dataset contains 40 GB of different images of Linux distributions in qcow2 format. The Linux distributions include Arch, Debian, and Fedora OSes.
5) *VMB:* This dataset contains 79 GB backups of Linux virtual machines collected from different users. Each user may have one or more VM backups.

B. Throughput

In this section, we evaluate the throughput of different chunking approaches with the five datasets. For each chunking approach, we chunk the input data in batches. In each batch, we read 128 MB of data to main memory and then chunk these data. Fig. 11 shows the average throughput of chunking under different chunk sizes. Moreover, the standard deviation of these chunking approaches are rather low in three trials. Except for LeapCDC, the throughput of other chunking approaches keep stable for different chunk sizes. Rabin and TTTD show similar performance because TTTD is based on Rabin. They show the lowest throughput because of the costly calculation of Rabin Hashing. The throughput of LeapCDC grows with the chunk size slightly because a bigger chunk size implies that more windows w_{ij} should be verified for the cut-condition at each position i. Since the number of windows w_{ij} is the same as the jump-length, LeapCDC jumps over more input data and offers higher throughput when the chunk size becomes larger. However, as mentioned in Section III-C, leapCDC suffers from the cost of pseudo-transformation. Also, since the chunk size is tightly coupled with the number of windows, this inflexibility also limits LeapCDC's application scenarios. Compared with LeapCDC, JC improves the throughput by about 2× on average. JC shows the highest throughput among these chunking approaches because it can jump over about one half of the input during chunking.

C. CPU Utilization

In this section, we limit the chunking throughput to 1000 MB/s and measure the utilization of CPUs for different chunking approaches. Fig. 12 shows the results of the TAR dataset because other datasets show similar results. Except for LeapCDC, the CPU consumption of other chunking approaches are not sensitive to the chunk size. Rabin and TTTD can only achieve 300 MB/s throughput at most even when the CPU utilization is as high as 90%. In contrast, all of the other chunking approaches can achieve a throughput of 1000 MB/s. Thus, the difference of CPU utilization between Rabin, TTTD and other chunking approaches are not as high as their differences of throughput shown in Fig. 11. Because JC jumps over about one half of the input data during chunking, it can reduce the CPU utilization by 30% on average compared with AE, fastCDC and LeapCDC. JC also reduces CPU utilization by 45% compared with Rabin and TTTD.

D. Deduplication Ratio

In this section, we evaluate the deduplication ratio of different chunking approaches. The deduplication ratio is measured as $\frac{data\ volume\ deduplicated}{data\ volume\ of\ input}$. The results are shown in Table IV. FSC shows the lowest deduplicate ratio because it divides the input in fixed sizes regardless of the data content, and thus suffers from the boundary-shift problem. Except for LeapCDC, all CDC approaches achieve a similar deduplication ratio. LeapCDC only achieves a similar deduplication ratio to other CDC approaches for the DOC dataset under chunk sizes of 4 KB and 16 KB. It shows a much lower deduplication ratio in other cases due to the limited applicability of the randomly-generated pseudo-random transformation array, which consists of numbers in a normal distribution. Since the probability that the randomly-generated pseudo-random transformation array fits for a given dataset is only 10% [7], it is hard to find the optimal transformation array for all five datasets and three different chunk sizes.

E. Sensitivity to Jump-Length

In this section, we evaluate the impact of different jump-lengths j_s on the deduplication ratio. When we set cOnes to $log_2(c) - 1$, and c_{avg} to $2^{cOnes+1}$, according to (3), (6), (8) and

TABLE IV
DEDUPLICATION RATIO (%) OF FIVE DATASETS UNDER DIFFERENT CHUNK SIZES AND CHUNKING APPROACHES

Chunking Approaches	Dataset / Chunk Size														
	DOC			NEWS			TAR			VMI			VMB		
	4K	16K	64K	4K	16K	64K	4K	16K	64K	4K	16K	64K	4K	16K	64K
FSC [1]	3.1	2.9	2.8	16.2	9.4	3.8	26.5	8.6	2.1	19.0	18.6	17.9	57.5	45.8	38.4
Rabin [4], [32]	14.7	11.2	9.0	69.7	49.6	26.1	54.7	38.9	23.7	61.0	56.9	47.6	59.8	55.3	49.4
TTTD [5]	15.2	11.6	9.2	71.0	53.0	33.8	56.1	40.2	25.0	61.3	57.6	48.7	60.3	55.9	50.2
AE [6], [28]	14.3	10.8	8.6	70.3	49.2	36.5	53.8	38.3	23.2	60.5	55.9	45.5	55.9	49.4	40.1
Gear [13]	14.3	10.9	8.8	70.7	50.7	34.4	55.3	38.8	23.6	61.0	56.8	47.2	59.8	55.2	49.2
FastCDC [3]	15.0	10.8	8.7	71.2	54.1	33.8	54.6	38.6	23.1	61.0	56.6	46.6	59.7	55.0	48.8
LeapCDC [7]	14.0	10.2	5.7	58.2	41.3	0.4	48.7	28.6	2.9	58.7	47.7	5.8	54.2	45.2	16.5
JC	**15.0**	**10.9**	**8.3**	**70.8**	**54.9**	**34.9**	**54.9**	**38.5**	**23.4**	**60.7**	**56.2**	**46.3**	**59.7**	**55.3**	**49.2**

TABLE V
DEDUPLICATION RATIO AND THROUGHPUT OF JC UNDER DIFFERENT JUMP-LENGTH (j_s)

jOnes	11	10	9	8	7
Jump-Length	4096	1365	585	273	132
Dedup. ratio	49.4%	48.5%	48.4%	48.2%	47.8%
Throughput (GB/s)	3.32	3.23	3.20	3.20	3.17

(10), we get:

$$j_s = \frac{2^{jOnes+cOnes}}{2^{cOnes} - 2^{jOnes}} \tag{16}$$

If we keep $cOnes$ unchanged and adjust $jOnes$ according to (17), we can get a series of parameters that allow the average chunk size to be the same. We measure the deduplication ratio using the TAR dataset when $jOnes$ changes from 11 to 7. The chunk size is set to 8 KB, and $cOnes$ is set to 12. The experimental results for other datasets are similar. If we set the $jOnes$ too small, most fingerprints are able to satisfy the outer hash judgment (as shown in line 11 of Algorithm 1). These fingerprints require two conditional judgments, and thus slow down the throughput. Thus, we allow the $jOnes$ to be larger than 7. The results are shown in Table V. With the decrease of $jOnes$, the deduplication ratio declines slightly. The reason is that smaller $jOnes$ leads to more jumps which have a little impact on the deduplication ratio. The throughput also decreases slightly because more conditional judgments are needed.

F. Performance Improvement

To demonstrate the performance gain of JC more clearly, we measure the execution time of four stages for different deduplication approaches. In this experiment, all fingerprints are kept in DRAM. As shown in Fig. 13, the chunking stage consumes about 30%∼80% of the total execution time because the rolling hash has to calculate the fingerprint of each sliding window when it slides forward for each byte. Since the hashing stage calculates the hash code per chunk rather than per byte, and xxHash can achieve a throughput as high as 30 GB/s [22], it only accounts for 2%∼10% of the total execution time. Compared with AE and FastCDC, JC can reduce the total execution time by about 25%. This demonstrates the effectiveness and efficiency of JC for improving the performance of data deduplication.

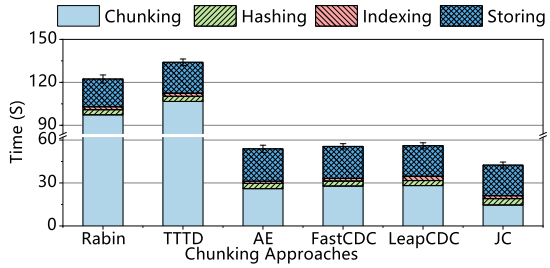


Fig. 13. Time spent in different stages of deduplication. *The standard deviations of the total execution time for different chunk approaches range from 1∼3 seconds.*

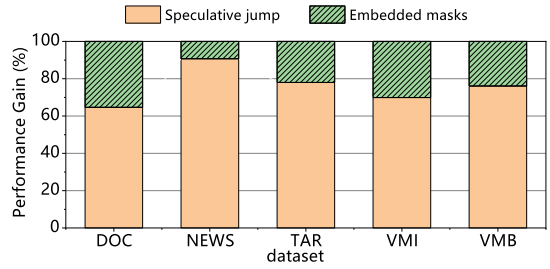
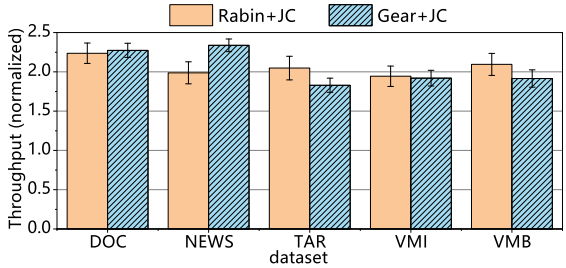


Fig. 14. Breakdown of performance gains in JC.

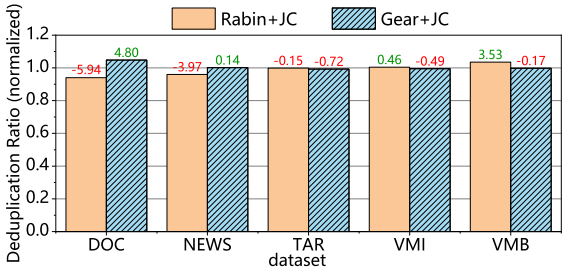
There are mainly two key techniques in JC, i.e., jump on specific conditions (Section III-B1) and embedded masks (Section III-B2). We evaluate these two techniques incrementally and present the breakdown of performance gains in Fig. 14. Jump and embedded masks contribute 76% and 24% performance gains on average, respectively. Jump achieves 91% contribution on the NEWS dataset. The reason is that the NEWS dataset is mainly composed of small web pages. When these small files are chunked, the sliding window is more likely to reach the end of the file after a jump. Since the chunking process involves less hash judgments, the embedded masks make less contribution on NEWS compared with other datasets.

G. Integration With Existing Techniques

1) Integration With Different Hash Algorithms: Although JC is implemented based on Gear for high efficiency, it can also be implemented with other hash algorithms such as Rabin. We evaluate the throughput and the deduplication ratio of JC based on those two hash algorithms, as shown in Fig. 15. JC improves the throughput of these two algorithms by about 2× because



(a) The throughput of JC based on Rabin and Gear, all normalized to the vanilla Rabin and Gear, respectively.



(b) The deduplication ratio of JC based on Rabin and Gear, all normalized to the vanilla Rabin and Gear, respectively.

Fig. 15. Impact of JC on throughput and deduplication for different hashing algorithms.

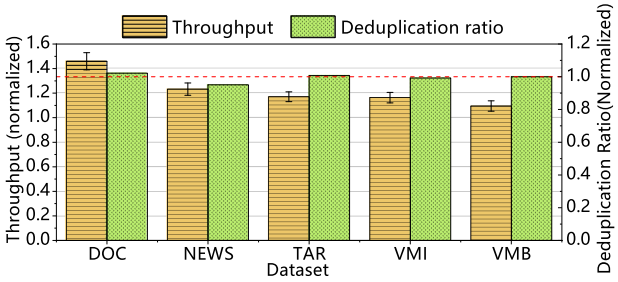


Fig. 16. Throughput and deduplication ratio of RapidCDC when JC is applied, all normalized to the vanilla RapidCDC. *The standard deviations of the normalized throughput are less than 0.07.*

it jumps over about 50% of the input data. JC also has a little impact on the deduplication ratio. Compared with the vanilla Rabin and Gear, the deduplication ratio of JC only shows -1.21% and +0.71% variations on average.

2) Integration With RapidCDC: RapidCDC leverages the locality of data duplication to reduce the cost of chunking. We integrate the key techniques of JC into RapidCDC, and then evaluate the performance improvement. RapidCDC exploits the historical information of chunking to predict the chunk size, and thus can reduce the cost of fingerprint calculations and hash judgment. When the key techniques of JC are applied to RapidCDC, JC can further improve the throughput of RapidCDC by 1.2× on average, as shown in Fig. 16. However, the "RapidCDC+JC" approach can remain similar deduplication ratios to the vanilla RapidCDC. This implies JC can improve the throughput of existing chunking schemes without compromising the deduplication ratio.

H. Overhead

As described in Section III-B2, JC needs to execute an additional condition judgement when a fingerprint satisfies the jump-condition. However, this condition is checked only at a probability of $1/2^{jOnes}$ (i.e., 0.05% and 0.02% for chunk sizes of 4 KB and 8 KB, respectively). Because of the low probability, the performance overhead caused by the extra condition judgement is negligible. This trivial computation overhead can be offset by a significant reduction of rolling hash calculation via speculative jumps. Moreover, since the JC algorithm only introduces a 8-byte constant–$maskJ$ in the main memory, it incurs trivial storage overhead relative to Rabin/Gear based CDC algorithms. Also, because JC does not incur a decline of the deduplication ratio, it has little impact on the storage efficiency.

V. Conclusion

This paper proposes a new chunking approach called JC to speed up the chunking process. JC reduces the cost of chunking by jumping over some input data under special conditions. Specifically, JC introduces a new condition called jump-condition and jumps over some input if the fingerprints of the sliding window satisfy this condition. Further, we embed the original cut-condition into jump-condition to speed up the chunking. We theoretically prove the efficiency of JC and its effectiveness on solving the boundary-shift problem. Our experimental result demonstrates that JC achieves 2× higher throughput while maintaining the same deduplication ratio compared with the state-of-the-art chunking algorithms.

References

[1] S. Quinlan and S. Dorward, "Venti: A new approach to archival data storage," in *Proc. Conf. File Storage Technol.*, 2002, pp. 1–13.
[2] W. Xia et al., "A comprehensive study of the past, present, and future of data deduplication," *Proc. IEEE*, vol. 104, no. 9, pp. 1681–1710, Sep. 2016.
[3] W. Xia et al., "FastCDC: A fast and efficient content-defined chunking approach for data deduplication," in *Proc. USENIX Annu. Tech. Conf.*, 2016, pp. 101–114.
[4] A. Z. Broder, "Some applications of Rabin's fingerprinting method," in *Sequences II: Methods in Communication, Security, and Computer Science*. Berlin, Germany: Springer, 1993, pp. 143–152.
[5] K. Eshghi and H. K. Tang, "A framework for analyzing and improving content-based chunking algorithms," Hewlett-Packard Labs, Palo Alto, CA, USA, Tech. Rep. 30, 2005.
[6] Y. Zhang et al., "AE: An asymmetric extremum content defined chunking algorithm for fast and bandwidth-efficient data deduplication," in *Proc. IEEE Conf. Comput. Commun.*, 2015, pp. 1337–1345.
[7] C. Yu, C. Zhang, Y. Mao, and F. Li, "Leap-based content defined chunking - Theory and implementation," in *Proc. IEEE Symp. Mass Storage Syst. Technol.*, 2015, pp. 1–12.
[8] D. Eastlake and P. Jones, "US secure hash algorithm 1 (SHA1)," IETF RFC 3174, 2001.
[9] R. Rivest, "The MD5 Message-Digest Algorithm," IETF RFC 1321, 1992.
[10] Z. Wang, S. Xia, and F. Zhang, "Further study on indecomposable cryptographic functions," *Front. Comput. Sci.*, vol. 17, no. 2, 2022, Art. no. 172803.
[11] Y. Huang, F. Zhang, Z. Liu, and H. Tian, "Pseudorandom number generator based on supersingular elliptic curve isogenies," *Sci. China Inf. Sci.*, vol. 65, no. 5, pp. 1–3, 2022.
[12] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," in *Proc. 18th ACM Symp. Operating Syst. Princ.*, 2001, pp. 174–187.

[13] W. Xia, H. Jiang, D. Feng, L. Tian, M. Fu, and Y. Zhou, "Ddelta: A deduplication-inspired fast delta compression approach," *Perform. Eval.*, vol. 79, pp. 258–272, 2014.
[14] F. Ni and S. Jiang, "RapidCDC: Leveraging duplicate locality to accelerate chunking in CDC-Based deduplication systems," in *Proc. ACM Symp. Cloud Comput.*, 2019, pp. 220–232.
[15] M. Fu, "Destor: An experimental platform for chunk-level data deduplication," 2014. [Online]. Available: https://github.com/fomy/destor
[16] M. Fu et al., "Design tradeoffs for data deduplication performance in backup workloads," in *Proc. 13th USENIX Conf. File Storage Technol.*2015, pp. 331–344.
[17] N. Bjørner, A. Blass, and Y. Gurevich, "Content-dependent chunking for differential compression, the local maximum approach," *J. Comput. Syst. Sci.*, vol. 76, no. 3/4, pp. 154–203, 2010.
[18] W. Xia, D. Feng, H. Jiang, Y. Zhang, V. Chang, and X. Zou, "Accelerating content-defined-chunking based data deduplication by exploiting parallelism," *Future Gener. Comput. Syst.*, vol. 98, pp. 406–418, 2019.
[19] P. Hamandawana, A. Khan, C.-G. Lee, S. Park, and Y. Kim, "Crocus: Enabling computing resource orchestration for inline cluster-wide deduplication on scalable storage systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1740–1753, Aug. 2020.
[20] M. Ajdari, W. Lee, P. Park, J. Kim, and J. Kim, "FIDR: A scalable storage system for fine-grain inline data reduction with efficient memory handling," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2019, pp. 239–252.
[21] P. Hamandawana, A. Khan, J. Kim, and T.-S. Chung, "Accelerating ML/DL applications with hierarchical caching on deduplication storage clusters," *IEEE Trans. Big Data*, vol. 8, no. 6, pp. 1622–1636, Dec. 2022.
[22] "xxHash," 2023. Accessed: Jun. 2023 [Online]. Available: http://www.xxhash.com
[23] "Intel processors and processor cores based on golden cove microarchitecture instruction throughput and latency," Feb. 2020. Accessed: Jun. 2023. [Online]. Available: https://cdrdv2.intel.com/v1/dl/getContent/723498
[24] "GCC, the GNU compiler collection," 2023. Accessed: Jun. 2023. [Online]. Available: https://gcc.gnu.org/
[25] "Vim - The ubiquitous text editor," 2023. Accessed: Jun. 2023. [Online]. Available: https://www.vim.org/
[26] W. Xia et al., "The design of fast content-defined chunking for data deduplication based storage systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 9, pp. 2017–2031, Sep. 2020.
[27] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Trans. Storage*, vol. 7, no. 4, pp. 1–20, 2012.
[28] Y. Zhang et al., "A fast asymmetric extremum content defined chunking algorithm for data deduplication in backup storage systems," *IEEE Trans. Comput.*, vol. 66, no. 2, pp. 199–211, Feb. 2017.
[29] N. Elias, P. Shilane, S. Sheinvald, and G. Yadgar, "DedupSearch: Two-phase deduplication aware keyword search," in *Proc. 20th USENIX Conf. File Storage Technol.*, 2022, pp. 233–246.
[30] Y. Zhang et al., "Improving restore performance of packed datasets in deduplication systems via reducing persistent fragmented chunks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 7, pp. 1651–1664, Jul. 2020.
[31] Y. Zhang, W. Xia, D. Feng, H. Jiang, Y. Hua, and Q. Wang, "Finesse: Fine-grained feature locality based fast resemblance detection for post-deduplication delta compression," in *Proc. 17th USENIX Conf. File Storage Technol.*, 2019, pp. 121–128.
[32] M. Rabin, "Fingerprinting by random polynomials," Center Res. Comput. Techn., Aiken Computation Lab., Tech. Rep. TR-15-81, 1981.

Xiaozhong Jin received the bachelor's degree from North China Electric Power University, China, in 2019. He is currently working toward the PhD degree with the Huazhong University of Science and Technology, China. His research interests include data deduplication and storage systems.

Haikun Liu (Member, IEEE) received the PhD degree in computer science and technology from Huazhong University of Science and Technology, in 2012. He is currently a professor with the School of Computer Science and Technology, Huazhong University of Science and Technology, China. He has co-authored more than 60 papers in most prestigious conferences and Journals, such as ASPLOS/HPCA/DAC/SC/ICS/HPDC, and TC/TPDS/TACO. His research interests include in-memory computing, virtualization technologies, cloud computing, and distributed systems. He is a senior member of CCF.

Chencheng Ye (Member, IEEE) received the PhD degree in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2019. He is currently an Associate Professor with the School of Computer Science and Technology, Huazhong University of Science and Technology. His research interests include hardware, and programming language supports for memory systems, with an emphasis on improving locality, programmability, and persistency.

Xiaofei Liao received the PhD degree in computer science and engineering from HUST, China, in 2005. He is currently a professor with the School of Computer Science and Technology, Huazhong University of Science and Technology (HUST), China. His research interests include computer architecture, system software, and Big Data processing. He was the recipient of the Excellent Youth Award from the National Science Foundation of China in 2018, and CCF-IEEE CS Young Computer Scientist Award in 2017. He is a member of IEEE Computer Society.

Hai Jin received the PhD degree in computer engineering from the Huazhong University of Science and Technology (HUST), in 1994. He is currently a chair professor of computer science and engineering, HUST, China. He was with The University of Hong Kong between 1998 and 2000, and as a visiting scholar with the University of Southern California between 1999 and 2000. He has co-authored more than 20 books and authored or coauthored more than 900 research papers. His research interests include computer architecture, parallel and distributed computing, Big Data processing, data storage, and system security. In 1996, he was awarded a German Academic Exchange Service fellowship to visit the Technical University of Chemnitz in Germany. He was the recipient of the Excellent Youth Award from the National Science Foundation of China in 2001. Jin is a Fellow of CCF, and a life member of the ACM.

Yu Zhang received the PhD degree in computer science from the Huazhong University of Science and Technology (HUST) in 2016. He is currently a associated professor with the School of Computer Science and Technology, HUST. His research interests include Big Data processing, graph computing and distributed systems. His current research focuses on application-driven Big Data processing and optimizations.